

JOBSHEET WEEK 3

WEB PROGRAMMING



SIMPUS-Mini

Responsive Design & UI/UX Design

DEPARTMENT OF INFORMATION TECHNOLOGY
STATE POLYTECHNIC OF MALANG
2026

Table of Contents

PART I — Jobsheet 3: Responsive Design	4
Jobsheet 3 Introduction	5
What's New in Jobsheet 3?	5
Folder Structure	5
1. Core Concepts of Responsive Design	6
1.1 What Is Responsive Web Design?	6
1.2 <code><meta name="viewport"></code> , Stopping Mobile Browsers from "Lying"	6
1.3 Media Queries: CSS That Asks First	6
1.4 Breakpoints: The Threshold Values	7
1.5 The "Desktop-First" Approach Used in This Jobsheet	7
2. What Changed in the HTML Files?	8
2.1 <code><meta viewport></code>	8
2.2 The Checkbox + Label Pair for the Hamburger Menu	8
2.3 The <code><div class="table-responsive"></code> Wrapper	8
3. CSS: Hamburger Menu with the Checkbox Hack	10
3.1 The CSS Involved	10
3.2 Step 1: Hide the Real Checkbox	10
3.3 Step 2: The Label Acts as the Button	11
3.4 Step 3: Hide the Menu on Narrow Screens (First)	11
3.5 Step 4: Sibling Combinator, Linking State to <code>nav</code>	11
3.6 Why Are These Rules Repeated Inside the Media Query?	12
3.7 A Vertical Menu When Open	13
4. CSS: A Scrollable Table (<code>table-responsive</code>)	14
4.1 The CSS	14
4.2 Two Ways to Handle Wide Tables on Narrow Screens	14
4.3 Why Wrap It in a <code><div></code> ?	14
4.4 The <code>auto</code> Value of <code>overflow-x</code>	14
4.5 When Does This Effect Actually Show?	15
5. CSS: Media Queries & Breakpoints	16
5.1 The Complete CSS	16
5.2 Breakpoint 1: Tablet (768px and Below), 3 Columns Become 2	16
5.3 Breakpoint 2: Mobile ($\leq 480\text{px}$): The Most Changes	17
5.4 The Statistic Card Grid on Mobile: 1 Column	17
5.5 Form Inputs at Full Width on Mobile	18
5.6 How to Test It Yourself in the Browser	18
6. Summary & Exercises	19
6.1 Summary of Jobsheet 3	19
6.2 Core Concepts to Remember	19
6.3 Try It Yourself	19
6.4 Exercises	20
PART II — Jobsheet 4: UI/UX Design	21
Jobsheet 4 Introduction	22

Folder Structure.....	22
1. Core Concepts of UI/UX Design.....	23
1.1 What Is the Difference Between UI and UX?	23
1.2 Why Design First?	23
1.3 Two Design Tools: Wireframes & User Flows	23
2. How to Read a Wireframe	24
2.1 Example: The Login Page Wireframe	24
2.2 Rules for Reading the Symbols	24
2.3 A More Complex Wireframe: The Librarian Dashboard	25
2.4 Why Are Wireframes Deliberately Plain?	25
3. User Flow: Borrowing & Returning	26
3.1 What Is a User Flow?	26
3.2 User Flow: Borrowing a Book.....	26
3.3 User Flow: Returning a Book.....	26
3.4 Why Write the User Flow Before the Form Exists?.....	27
4. Actors & Their Link to Authorisation	28
4.1 What Is an "Actor" in UI/UX Design?	28
4.2 The "Guest" Actor, What You Can Already Try	28
4.3 The "Librarian" Actor: Newly Designed in This Jobsheet	28
4.4 What Is Authorisation?	28
4.5 Why Define Actors First?	28
5. Connections to the Existing Code.....	30
5.1 The Official Note from <code>wireframe.md</code>	30
5.2 Colours & Styles Stay the Same, No Need to Build CSS from Scratch.....	30
5.3 The Navbar Will Gain a Menu, but the CSS Pattern Stays.....	30
5.4 The Same Statistic Cards, a Different Context	31
5.5 Edge Cases Already Recorded	31
6. Summary & Exercises	32
6.1 Summary of Jobsheet 4.....	32
6.2 Core Concepts to Remember.....	32
6.3 How to Try This Jobsheet.....	32
6.4 Exercises.....	32

PART I

Jobsheet 3: Responsive Design

Building one web page whose layout adapts to phone, tablet and desktop screens: the viewport meta tag, a hamburger menu without JavaScript, scrollable tables, and media queries.

Jobsheet 3 Introduction

This documentation continues from the jobsheet-02 documentation and is written for students who are new to HTML/CSS. If you are not yet comfortable with basic HTML (jobsheet-01) or basic CSS, Flexbox, Grid, the box model (jobsheet-02), read the earlier documentation first, because the chapters here refer back to it often.

What's New in Jobsheet 3?

As noted in this jobsheet's README.md, there are 4 additions compared to jobsheet-02:

1. The `<meta name="viewport">` tag on every page.
2. A **hamburger** menu (the ☰ icon) on narrow screens, built purely with CSS using the **checkbox hack** (no JavaScript at all).
3. Tables wrapped in `<div class="table-responsive">` so they can be scrolled sideways on narrow screens, instead of squeezing the columns until they become unreadable.
4. **Media queries** in `style.css` that change the statistic card grid from 3 columns → 2 columns → 1 column as the screen width changes.

All of these changes belong to the broader topic of **Responsive Web Design**, building a single web page whose layout adapts automatically to different screen sizes (phone, tablet, laptop, large monitor), without having to build a separate page for each device.

Folder Structure

```
jobsheet-03/
├── index.html
├── assets/
│   └── css/
│       └── style.css      # Checkbox hack + media queries added
├── books/
│   ├── list.html         # Table wrapped in .table-responsive
│   └── add.html
├── members/
│   ├── list.html         # Table wrapped in .table-responsive
│   └── add.html
├── README.md
└── Documentation/        # This documentation folder
```

1. Core Concepts of Responsive Design

Before dissecting the code, get to know the big terms behind the "responsive look" that will keep appearing in the following chapters.

1.1 What Is Responsive Web Design?

Responsive Web Design (RWD) is an approach to building web pages so that **the layout adapts automatically** to the screen width of the device being used, phone, tablet, laptop or large monitor, using **one** set of HTML/CSS files, without having to build a separate version for each device (for example the `m.site.com` phone-only address that was popular before the RWD era).

Before this jobsheet, `index.html` and its siblings already "worked" on a phone (the browser still displayed them), but they were not yet **responsive**, because without `<meta name="viewport">` (discussed in §1.2), a mobile browser assumes the page was designed for a desktop screen and then zooms the whole page out so it "fits" the small screen. The result: tiny text, and the user has to pinch-zoom manually.

1.2 `<meta name="viewport">`, Stopping Mobile Browsers from "Lying"

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

This is added to the `<head>` of every page in this jobsheet (see the detailed change in chapter 2). Without this line, a mobile browser **pretends** to have a screen width of roughly 980px (called the *default layout viewport*) and then zooms the rendered result out so it fits the much narrower real screen, which is exactly why so many older sites look "miniature" and need manual zooming on a phone.

The `content` attribute holds two instructions:

Instruction	What it means
<code>width=device-width</code>	The viewport width (the display area) is set equal to the physical screen width of the device, rather than the 980px default.
<code>initial-scale=1</code>	The initial zoom level is set to 1:1 (neither zoomed out nor in) when the page first loads.

With these two instructions, 1 CSS pixel (the `px` you write in `style.css`) corresponds closely to what is actually seen on the phone screen, so **media queries** (discussed next) can work accurately based on the real screen width.

1.3 Media Queries: CSS That Asks First

A **media query** is a CSS feature that lets a set of CSS rules apply **only if a certain condition is met**, most commonly, the condition is the screen (viewport) width. The syntax:

```
@media (max-width: 768px) {  
  /* the CSS rules here apply ONLY  
  if the screen is 768px wide or narrower */  
}
```

```
}
```

- `@media`, the keyword that opens a media query block.
- `(max-width: 768px)`, the **condition**: it applies as long as the viewport width is **less than or equal to** 768px.
- All the CSS rules inside the curly braces `{ }` are only "active" while that condition is true. As soon as the screen is made wider than 768px (for example, by resizing the browser window), the rules inside stop applying automatically, no JavaScript and no page reload needed, purely a CSS capability.

1.4 Breakpoints: The Threshold Values

A **breakpoint** is the particular screen width used as a "dividing line" in a media query. This jobsheet has 2 breakpoints:

Breakpoint	Value	Roughly Represents
Tablet	768px	Tablets and other narrow screens
Mobile	480px	Phones

These numbers are **not a universal rule**, many projects choose different breakpoints depending on their design needs. 768px and 480px are simply common conventions that sit close to typical tablet and phone widths.

1.5 The "Desktop-First" Approach Used in This Jobsheet

There are two common strategies for writing responsive CSS:

Strategy	How It Works
Desktop-first	Write the default styles for large screens first, then override some of them inside <code>@media (max-width: ...)</code> for narrower screens.
Mobile-first	The opposite: write the default styles for narrow screens first, then add more styles inside <code>@media (min-width: ...)</code> for wider screens.

This jobsheet uses the **desktop-first** approach: notice that all the styles from jobsheet-02 (the 3-column grid, the horizontal navbar and so on) remain the **default/base** styles, and `style.css` in this jobsheet merely **adds** two `@media (max-width: ...)` blocks at the very **bottom** of the file to override them on narrow screens (see chapter 5).

Why must they go at the bottom of the file? Because when two CSS rules have the same selector specificity (remember the specificity concept from the jobsheet-02 documentation), the rule written **later** in the file wins. If the `@media` block were placed **before** the default styles, the default styles written afterwards would override the media query result, so the responsive effect would "disappear" without you meaning it to. This is a common beginner mistake when adding media queries to an existing stylesheet for the first time.

2. What Changed in the HTML Files?

Just like the jobsheet-01 → jobsheet-02 transition, the overall HTML structure in jobsheet-03 **has not changed** from jobsheet-02. There are 3 small but important additions to the HTML, plus several new lines of pure CSS in `style.css` (covered from chapter 3 onwards).

2.1 `<meta viewport>`

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Added to the `<head>` of **all 5** HTML pages, immediately below `<meta charset="UTF-8">`. The full explanation of why this line matters is in core concepts §1.2.

2.2 The Checkbox + Label Pair for the Hamburger Menu

```
<header>
  <h1>SIMPUS-Mini</h1>
  <input type="checkbox" id="nav-toggle" class="nav-toggle">
  <label for="nav-toggle" class="nav-toggle-label">&#9776;</label>
  <nav>
    ...
  </nav>
</header>
```

Two new elements appear inside `<header>`, **before** `<nav>`:

- `<input type="checkbox" id="nav-toggle" class="nav-toggle">`, an ordinary checkbox. But through CSS (covered in chapter 3), this checkbox will be **hidden** from view, it is used not to be ticked visually, but simply to **store the state** "is the menu open or closed" (ticked = open).
- `<label for="nav-toggle" class="nav-toggle-label">☰</label>`, remember from the jobsheet-01 documentation that the `for="nav-toggle"` attribute **links** this label to the checkbox whose `id` is `nav-toggle`. The effect: **clicking this label is the same as clicking the checkbox itself**, and that is the basis of the "checkbox hack" explained in more detail in chapter 3.
- `☰`; this is an **HTML entity** written as a numeric code (similar to `©` or `—`, already covered in the jobsheet-01 documentation), which displays the symbol  (three horizontal bars, the "hamburger" icon commonly used for menus on narrow screens).

Important: all three of these elements (`h1`, `input`, `label`) sit **directly inside** `<header>`, as siblings of `<nav>`, not inside `<nav>`. This arrangement is deliberate, so that the CSS can use the **sibling combinator** (`~`) between the checkbox and `<nav>`; the mechanism is explained in detail in chapter 3.

2.3 The `<div class="table-responsive">` Wrapper

In `books/list.html` and `members/list.html`, the table that previously stood on its own (see the jobsheet-01 documentation) is now wrapped in one extra `<div>`:

```
<div class="table-responsive">
  <table>
    ...
  </table>
```

```
</div>
```

The `<div>` here is used purely as a **technical wrapper** for styling purposes (`overflow-x: auto`, covered in chapter 4), unlike the semantic tags (`header`, `section`, `article` and so on from the jobsheet-01 documentation) which carry meaning about the content. A `<div>` **deliberately has no** semantic meaning, it is just a "plain box" used when we need a CSS/JS target without needing a specific tag. The class name `table-responsive` is one **we made up ourselves** (not a built-in HTML name), chosen so its purpose is easy to read: "a table that has been made responsive".

3. CSS: Hamburger Menu with the Checkbox Hack

A menu that can be opened and closed by clicking, **without a single line of JavaScript**. The trick is called the **checkbox hack**, and it makes use of the `:checked` pseudo-class together with the CSS **sibling combinator**.

3.1 The CSS Involved

This code sits in 2 different places in `style.css`: the base styles (always active) and the styles inside the mobile media query (active only on narrow screens).

Base styles (outside the media query):

```
/* ===== Hamburger Menu (checkbox hack) ===== */
.nav-toggle {
  display: none;
}

.nav-toggle-label {
  display: none;
  font-size: 1.6rem;
  color: #fff;
  cursor: pointer;
}
```

Inside @media (max-width: 480px) (covered fully in chapter 5):

```
header {
  position: relative;
}

.nav-toggle-label {
  display: block;
}

header nav {
  display: none;
  width: 100%;
  order: 3;
  margin-top: 1rem;
}

.nav-toggle:checked ~ nav {
  display: block;
}

header nav ul {
  flex-direction: column;
  gap: 0.75rem;
}
```

3.2 Step 1: Hide the Real Checkbox

```
.nav-toggle {
  display: none;
}
```

The actual HTML checkbox (the little tickable box) is **hidden completely** with `display: none`. Remember from its HTML (chapter 2 §2.2) that this checkbox was never meant to be seen, it is used

only as a hidden "state holder": **ticked = menu open, unticked = menu closed**. That state is what other CSS later "reads" through the `:checked` pseudo-class in §3.5.

3.3 Step 2: The Label Acts as the Button

```
.nav-toggle-label {  
  display: none;      /* hidden on large screens */  
  font-size: 1.6rem;  
  color: #fff;  
  cursor: pointer;  
}
```

Because the real checkbox is hidden, the `<label>` (holding the `≡` icon from chapter 2) is what appears in its place as the "button" the user can click. Remember that the relationship between `<label for="nav-toggle">` and `<input id="nav-toggle">` was already explained in the jobsheet-01 documentation: **clicking the label = clicking its checkbox**, even though the checkbox itself is not visible on screen at all. `cursor: pointer` signals that this label is clickable (just like an ordinary button, see the jobsheet-02 documentation §7.6).

Notice that `.nav-toggle-label` is given `display: none` in the base styles, meaning that on **large** screens (desktop/tablet) this hamburger icon **does not appear at all**, because the navigation menu is already visible in a horizontal row (see the jobsheet-02 documentation). The icon is only revealed (`display: block`) inside `@media (max-width: 480px)`, the flow is explained in §3.6.

3.4 Step 3: Hide the Menu on Narrow Screens (First)

```
header nav {  
  display: none;  
  width: 100%;  
  order: 3;  
  margin-top: 1rem;  
}
```

This rule applies **only inside** `@media (max-width: 480px)`, on narrow screens, `<nav>` (the full navigation menu) is hidden **by default**, so it does not fill up the small phone screen. `order: 3` is a Flexbox property (remember that `<header>` is a flex container from the jobsheet-02 documentation §4.3), it makes `<nav>` **always appear last/at the bottom** among the other flex items (`h1`, the hidden checkbox, the label) when it is eventually displayed, rather than being wedged in the middle.

3.5 Step 4: Sibling Combinator, Linking State to `nav`

```
.nav-toggle:checked ~ nav {  
  display: block;  
}
```

This is the **heart** of the whole checkbox hack. Let's take the selector apart piece by piece:

Piece	What it means
<code>.nav-toggle</code>	The element with the class <code>nav-toggle</code> , that is, our checkbox.
<code>:checked</code>	A pseudo-class (remember from the jobsheet-02 documentation §1.4): it applies only when that element is in the ticked state.

Piece	What it means
~	The general sibling combinator , a new selector not covered in earlier jobsheets. It means "select the <code>nav</code> element that is a sibling of <code>.nav-toggle</code> and appears after it in the HTML."
<code>nav</code>	The target element whose style is changed.

So the complete selector `.nav-toggle:checked ~ nav` means: "if the `.nav-toggle` checkbox is currently ticked, change the style of the `<nav>` element that sits alongside it, after it." This is exactly why the HTML arrangement in chapter 2 §2.2 matters: `input`, `label` and `nav` must all sit **directly** inside `<header>` (siblings of one another), not with `nav` nested inside another element, the sibling combinator **only** works between elements at the same level.

The complete flow:

1. The user clicks the label (the ☰ icon).
2. Because of `<label for="nav-toggle">`, that click **ticks** the hidden `#nav-toggle` checkbox.
3. The checkbox now matches `:checked`.
4. The selector `.nav-toggle:checked ~ nav` becomes active → `<nav>` (previously `display: none` from §3.4) becomes `display: block` → the menu **appears**.
5. Click the label again → the checkbox is no longer ticked → the `:checked ~ nav` rule no longer applies → `<nav>` returns to `display: none` from its base rule → the menu is **hidden again**.

All of this state switching is handled purely by the browser through CSS, without any event listener or JavaScript code, which is why the technique is called the "checkbox hack": it **borrow**s the checkbox's built-in behaviour (it can be ticked or not, and it has a `:checked` pseudo-class) for a purpose well outside the usual form context.

3.6 Why Are These Rules Repeated Inside the Media Query?

```
header {
  position: relative;
}

.nav-toggle-label {
  display: block;
}
```

Notice that `.nav-toggle-label` appears **twice** in `style.css`: once in the base styles (`display: none;`, §3.3) and again inside `@media (max-width: 480px) (display: block;)`. This is **not** a mistake or duplication, it is the standard pattern in desktop-first responsive CSS (remember core concepts §1.5): the base styles hide the hamburger icon on large screens, then the media query **overrides them back** to visible specifically when the screen is $\leq 480\text{px}$. Because the rule inside `@media` is written **after** the base style in the file (and both have the same specificity, a single class selector each), the rule inside the media query wins whenever its condition is met.

`header { position: relative; }` is likewise only activated at this mobile breakpoint, the `position: relative` property is needed so that the `<nav>` element (which will be given `width: 100%` when open) measures its width relative to `<header>`, rather than to some other outer element that might be a different size.

3.7 A Vertical Menu When Open

```
header nav ul {  
  flex-direction: column;  
  gap: 0.75rem;  
}
```

Remember from the jobsheet-02 documentation that `header nav ul` has been a flex container (`display: flex`) since jobsheet-02, laying the menu items out **horizontally** by default. The `flex-direction: column` property inside this mobile media query changes the direction of the flex items to **vertical**, which makes sense, because on a narrow phone screen an open menu is more comfortable to read as a stacked list than as a horizontal row where each item would be far too cramped.

4. CSS: A Scrollable Table (`table-responsive`)

This short chapter covers the solution to a classic problem: a table with many columns looks very cramped and becomes hard to read if it is forced to fit on a phone screen.

4.1 The CSS

```
/* ===== Responsive Table ===== */
.table-responsive {
  overflow-x: auto;
}
```

Just **one property**, but enough to solve the problem.

4.2 Two Ways to Handle Wide Tables on Narrow Screens

There are two common strategies for dealing with wide tables on narrow screens:

1. **Squeeze the table contents**, force all the columns to shrink so they fit the narrow screen. The problem: the text ends up truncated or unreadable, especially in columns like "Author" that hold long names.
2. **Leave the table at its natural width, but let it scroll sideways**, this is the approach chosen in this jobsheet. The column widths stay comfortable to read, and the user simply swipes (on a phone) or scrolls horizontally (with a trackpad or mouse) to see the columns that are not yet visible.

4.3 Why Wrap It in a `<div>`?

Remember from chapter 2 §2.3 that `<table>` is wrapped in an extra `<div class="table-responsive">`. This is because `overflow-x: auto` **cannot be applied directly** to a `<table>` element as effectively as it can to a wrapping `<div>`:

- A wide `<table>` (one with many columns) will simply **push its own containing box wider**, rather than being clipped and triggering a scrollbar on itself.
- By wrapping it in a `<div class="table-responsive">` that carries `overflow-x: auto`, it is this `<div>` that has a **fixed width** (following its containing `<section>` box, see the jobsheet-02 documentation §5.3), while the `<table>` inside is free to grow as wide as it likes. If the `<table>` becomes wider than its wrapping `<div>`, only then does `overflow-x: auto` show a **horizontal scrollbar** automatically, confined to the table area alone, rather than scrolling the whole page.

4.4 The `auto` Value of `overflow-x`

```
overflow-x: auto;
```

- `overflow-x` controls the behaviour of content that exceeds the width of an element, **specifically in the horizontal direction** (there is also `overflow-y` for the vertical direction, and `overflow` for both at once).

- The value `auto` means: the browser **only shows** a scrollbar if one is actually needed (the content inside exceeds the width of the box). If the table is narrow enough to fit (on a wide desktop screen, for instance), no scrollbar appears at all, behaviour that is friendlier than the value `scroll`, which would **always** show a scrollbar even when it is not needed.

4.5 When Does This Effect Actually Show?

Unlike the hamburger menu (chapter 3), which only becomes active at the mobile breakpoint, this `.table-responsive` rule is **always active** at every screen size (it is not wrapped in `@media` at all), it is just that its effect is **not visible** on wide screens, because the table is already narrow enough to fit without scrolling. The horizontal scroll effect only "appears" once the screen width (and therefore the width of the wrapping `<section>` from the jobsheet-02 documentation) becomes narrower than the total width of the table (the 5 columns of the Book List, or the 5 columns of the Member List).

5. CSS: Media Queries & Breakpoints

This chapter covers the two `@media` blocks at the very **bottom** of `style.css`, where most of the responsive "magic" in this jobsheet actually happens.

5.1 The Complete CSS

```
/* ===== Responsive Breakpoints ===== */

/* Tablet and below */
@media (max-width: 768px) {
  main section:nth-of-type(2) {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Mobile */
@media (max-width: 480px) {
  header {
    position: relative;
  }

  .nav-toggle-label {
    display: block;
  }

  header nav {
    display: none;
    width: 100%;
    order: 3;
    margin-top: 1rem;
  }

  .nav-toggle:checked ~ nav {
    display: block;
  }

  header nav ul {
    flex-direction: column;
    gap: 0.75rem;
  }

  main section:nth-of-type(2) {
    grid-template-columns: 1fr;
  }

  form input,
  form select {
    max-width: 100%;
  }
}
```

5.2 Breakpoint 1: Tablet (768px and Below), 3 Columns Become 2

```
@media (max-width: 768px) {
  main section:nth-of-type(2) {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

Remember from the jobsheet-02 documentation that the selector `main section:nth-of-type(2)` targets the "Summary" section (the Total Books / Total Members / On Loan statistic cards) on the

Home page, whose base style (outside any media query) is `grid-template-columns: repeat(3, 1fr)`, 3 equal columns. Inside this breakpoint, that value is **overridden** to `repeat(2, 1fr)`, 2 columns. Because the selector is exactly identical to the base style (both `main section:nth-of-type(2)`, the same specificity), the rule inside `@media`, written later in the file, wins for as long as the screen width condition holds (remember that writing order matters, already discussed in core concepts §1.5).

The result: on a tablet screen ($\leq 768\text{px}$), the 3 statistic cards that used to sit side by side in one row are now laid out in 2 columns (the third card automatically drops to a second row), CSS Grid handles this new row automatically as soon as the number of columns is reduced.

5.3 Breakpoint 2: Mobile ($\leq 480\text{px}$): The Most Changes

This second block is far busier, because a phone screen needs the most adjustments:

Rule	Effect
<code>header { position: relative; }</code>	The positioning basis for the opened <code><nav></code> .
<code>.nav-toggle-label { display: block; }</code>	The <code>≡</code> hamburger icon is revealed.
<code>header nav { display: none; ... }</code>	The menu is hidden by default on mobile screens.
<code>.nav-toggle:checked ~ nav { display: block; }</code>	The menu is revealed when the hamburger is clicked .
<code>header nav ul { flex-direction: column; ... }</code>	The menu items stack vertically when open.
<code>main section:nth-of-type(2) { grid-template-columns: 1fr; }</code>	The statistic cards become 1 column (fully stacked vertically).
<code>form input, form select { max-width: 100%; }</code>	Form inputs stretch to full width.

5.4 The Statistic Card Grid on Mobile: 1 Column

```
main section:nth-of-type(2) {  
  grid-template-columns: 1fr;  
}
```

The value `1fr` (only **one** fractional part) means this grid now has only **1 column**, filling all the available space, effectively making the three statistic cards (Total Books, Total Members, On Loan) **stack vertically**, one below the other, each as wide as the screen. Here is the full progression across the 3 breakpoints:

Screen Width	<code>grid-template-columns</code>	Card Layout
Desktop ($>768\text{px}$)	<code>repeat(3, 1fr)</code> (base style, jobsheet-02)	3 columns side by side
Tablet ($\leq 768\text{px}$)	<code>repeat(2, 1fr)</code> (§5.2)	2 columns, third card drops down
Mobile ($\leq 480\text{px}$)	<code>1fr</code>	1 column, all stacked vertically

On a narrow screen such as a phone, 3 cards side by side would leave each card too cramped to display its large number comfortably (remember the `1.8rem` font size on the statistic numbers from the jobsheet-02 documentation §6.6), stacking them vertically gives each card the full width of the screen.

5.5 Form Inputs at Full Width on Mobile

```
form input,  
form select {  
  max-width: 100%;  
}
```

Remember from the jobsheet-02 documentation §8.4 that the base style for inputs and selects is `width: 100%; max-width: 400px;`, full width, but **capped** at 400px so they do not stretch excessively on a roomy desktop screen. At this mobile breakpoint that `max-width` cap is **relaxed** to `100%`, on a phone screen, which is already narrow by nature (rarely more than 400px), the 400px cap is no longer needed and could even leave an awkward empty gap to the right of the input if the screen is slightly wider than 400px. Notice that the `width: 100%` property from the base style **does not need repeating** here, only `max-width` needs overriding, because `width: 100%` already applies from the base style and does not conflict with the purpose of this breakpoint.

5.6 How to Test It Yourself in the Browser

1. Open `index.html` (or any other page) in the browser.
2. Open **DevTools** (right-click → *Inspect*, or the `F12` key).
3. Turn on **responsive/device mode** (in Chrome: the phone/tablet icon in the top-left corner of the DevTools panel, or `Ctrl+Shift+M`).
4. Change the screen width gradually from wide (>768px) to narrow (<480px) and observe:
 - The statistic cards change from 3 → 2 → 1 column exactly as you cross 768px and 480px.
 - The navigation menu (in the header) turns into a ☰ icon as soon as the width is ≤480px.
 - Click that ☰ icon, the menu opens vertically below the header (the checkbox hack effect from chapter 3).
 - Open `books/list.html` and narrow the screen until the table is wider than the screen, then try scrolling that table sideways.

6. Summary & Exercises

6.1 Summary of Jobsheet 3

Section	Code Location	Concepts Learned
Core Concepts	—	Responsive design, <code><meta viewport></code> , media queries, breakpoints, the desktop-first strategy
HTML Changes	All <code>.html</code> files	The viewport tag, the <code>input[checkbox]+label</code> pair, <code><div class="table-responsive"></code>
Hamburger Checkbox Hack	<code>.nav-toggle, .nav-toggle-label, .nav-toggle:checked ~ nav</code>	The <code>~</code> sibling combinator, the <code>:checked</code> pseudo-class, pure-CSS interactivity without JavaScript
Responsive Table	<code>.table-responsive</code>	<code>overflow-x: auto</code> , the wrapper <code><div></code> pattern for horizontal scrolling
Media Queries & Breakpoints	<code>@media (max-width: 768px), @media (max-width: 480px)</code>	The <code>@media</code> syntax, tablet/mobile breakpoints, overriding based on writing order

6.2 Core Concepts to Remember

1. `<meta name="viewport">` is a **hard requirement** for responsive design, without it, none of the media queries in your CSS will work properly on a phone, because the mobile browser will assume a screen width of 980px (chapter 1).
2. **CSS can be made interactive without JavaScript**, by making use of built-in state pseudo-classes such as `:checked` (the checkbox hack) combined with the `~` sibling combinator (chapter 3). This is a useful technique for simple interactions, though for more complex cases JavaScript remains more flexible (as hinted in this jobsheet's README.md, it will be replaced by JavaScript in Jobsheet 5).
3. **`overflow-x: auto` on a wrapping `<div>`** is the standard pattern for handling wide tables or content on narrow screens without breaking the rest of the page layout (chapter 4).
4. **Media queries override base styles based on writing order** when selector specificity is equal, which is why the breakpoints in this jobsheet are deliberately placed at the very **bottom** of the file (chapter 5).
5. **Desktop-first vs mobile-first** are the two strategies for writing responsive CSS; this jobsheet uses desktop-first (`max-width`), the opposite of mobile-first (`min-width`), both are valid, and the choice depends on your team's or project's habits (chapter 1 §1.5).

6.3 Try It Yourself

1. Follow the DevTools testing steps in chapter 5 §5.6.

2. In DevTools responsive mode, try **changing the screen width slowly** while watching the statistic cards, notice how the layout "breaks" into a different number of columns exactly at 768px and 480px. This is the breakpoint effect working precisely at the point you specified.
3. Click the ☰ hamburger icon repeatedly in mobile mode and observe that the menu opens and closes **without reloading the page**, prove for yourself that this is pure CSS by opening the DevTools **Console** tab; no JavaScript log or error will be triggered by this interaction at all.

6.4 Exercises

1. **Add a new breakpoint**, for example `@media (min-width: 1400px)` for very wide monitors, changing `main { max-width: 1000px; }` (from the jobsheet-02 documentation) to something wider at this breakpoint only.
2. **Change the tablet breakpoint** from 768px to 900px, then observe at which screen width the card layout changes, proving that breakpoints can be adjusted freely to suit the design.
3. **Apply the table-responsive pattern** to another element that could grow too wide on a narrow screen, for instance, if you ever add a long `<pre>` code block to one of the pages.
4. **Move the hamburger icon**; for example, move `.nav-toggle-label` to the last position inside `<header>` (after `<nav>`), and then check whether the sibling combinator `.nav-toggle:checked ~ nav` from chapter 3 §3.5 still works. Remember the note that the `~` combinator requires the target to come **after** its source element in the HTML.
5. **Compare it with the mobile-first approach**, try rewriting `style.css` from scratch using `@media (min-width: ...)` instead of `max-width`, and feel the difference in the way you have to think.

If any part is still confusing, especially the sibling combinator and the checkbox hack, try rereading chapter 3 while experimenting directly in DevTools, this concept is far easier to grasp by clicking the hamburger icon while watching the DevTools **Elements** panel show and hide the `checked` attribute on the checkbox in real time.

PART II

Jobsheet 4: UI/UX Design

Designing before writing code: UI versus UX, how to read a wireframe, the borrowing and returning user flows, actors and authorisation, and the links back to the running code.

Jobsheet 4 Introduction

This jobsheet focuses on what is genuinely new: the **design process (UI/UX design)** set out in [docs/wireframe.md](#), a planning document for features that **have no code yet** (Login, Librarian Dashboard, Loans, Returns, History), which will only start to be implemented in the jobsheets that follow.

Before writing code for a reasonably complex new feature (Login, Loan/Return transactions), a developer first **designs** how that feature will look and how it will be used, so as not to write the code first and only then realise the flow is confusing or incomplete. That design process is called **UI/UX design**, and it is expressed in the form of **wireframes** and **user flows**, two terms explained fully in chapters 2 and 3.

Folder Structure

```
jobsheet-04/
├── index.html           # Exactly the same as jobsheet-03
├── assets/css/style.css # Exactly the same as jobsheet-03
├── books/               # Exactly the same as jobsheet-03
├── members/            # Exactly the same as jobsheet-03
├── docs/
│   └── wireframe.md     # NEW – the design for features not yet coded
├── Infographic.png
├── README.md
└── Documentation/       # This documentation folder
```

Because the HTML/CSS is identical to jobsheet-03, you do not need to reread the tag and CSS explanations, simply refer back to the jobsheet-01, jobsheet-02 and jobsheet-03 documentation if you need to refresh your memory.

1. Core Concepts of UI/UX Design

1.1 What Is the Difference Between UI and UX?

These two terms are often mentioned together, but they mean different things:

Term	Stands For	Focus
UI	<i>User Interface</i>	The look , colours, buttons, layout, typography. This is what you have been building since jobsheet-02 through <code>style.css</code> .
UX	<i>User Experience</i>	The flow and the feel , can the user complete their task easily, do the steps make sense, is anything confusing?

A simple analogy: if an application were a shop, **UI** is how the shelves and window displays are arranged so they look attractive; **UX** is whether customers can easily find what they came for and reach the till without getting lost. A shop can look beautiful (good UI) but be confusing to shop in (poor UX), or the other way round.

1.2 Why Design First?

Imagine writing the HTML for a "Borrow a Book" page with no design at all, you would have to decide all at once: which fields are needed, what order the steps go in, what happens if a book is out of stock, which page appears after the data is saved, and so on, all while typing HTML tags. It is very easy to forget an important condition (such as "a book with 0 stock cannot be borrowed") when designing and coding happen **at the same time**.

By producing a design first on paper or in text (like `docs/wireframe.md`), a developer can:

1. Think through **the whole flow** before being tied down by technical code details.
2. Find gaps and questions (for example: "what if the member still has overdue loans?") **before** writing code, while they are still cheap to change, changing one line of a design document is far cheaper than rewriting finished code.
3. Have a clear "map" to refer to when the real coding begins.

This is exactly why jobsheet-04 **deliberately adds no code**, all of this jobsheet's energy goes into designing first, in line with its stated learning outcome: "*Design the UI/UX of an application (project).*"

1.3 Two Design Tools: Wireframes & User Flows

`docs/wireframe.md` contains two kinds of design document that complement each other:

- **Wireframe**: a rough sketch of the layout of *a single page*, with no colour or font detail, showing only which elements exist and roughly where they sit. Covered in detail in chapter 2.
- **User flow**: a diagram showing the **order of movement** between pages or steps that a user goes through to complete one task (borrowing a book, for example). Covered in detail in chapter 3.

Where a wireframe answers "what is on this page?", a user flow answers "to get here, which steps does the user pass through?", you need to see both together to understand the design as a whole.

2. How to Read a Wireframe

The **wireframes** used in `docs/wireframe.md` are drawn as **ASCII art** (pictures made from ordinary text characters) inside markdown code blocks, similar to the approach once used for the box model diagram in the jobsheet-02 documentation before it was replaced with an SVG. The difference is that for a **design wireframe** (rather than a final, published diagram) ASCII art is exactly the right choice: quick to draw, easy to change when the design changes, and requiring no special design tool, ideal for the "rough sketch" stage before something is really built.

2.1 Example: The Login Page Wireframe

```
+-----+
| SIMPUS-Mini |
+-----+
|           |
| [ Librarian Login ] |
|           |
| Username : [ _____ ] |
| Password : [ _____ ] |
|           |
| [ Sign In ] |
|           |
| No account yet? Register here |
+-----+
```

2.2 Rules for Reading the Symbols

A text wireframe like this follows a simple set of agreed symbols:

- Boxes made from hash characters (+), dashes (-) and upright bars (read as "pipe"), these mark the **outer boundary** of a page, or of a box/panel inside it.
- A single upright bar in the middle of a box (a vertical divider), this marks a **boundary between sections** within one page, much like the `<section>` element you already know from the jobsheet-01 documentation.
- `[ _____ ]` (square brackets containing underscores), an **input box**, where the user types text (later becoming `<input type="text">` or `<input type="password">`; remember this concept from the jobsheet-01 documentation).
- `[ Text ]` (square brackets containing text), a clickable **button** (later becoming `<button>`).
- Plain text with no brackets, a static label or caption (later becoming `<label>`, `<h1>`, `<p>` and so on).

Translating the Login wireframe above into the HTML terms you already know:

- `SIMPUS-Mini` on the top line → will become `<header><h1>` as on every existing page.
- `Librarian Login` → a section heading, much like the `<h2>` on the other pages (for example "Book List" in the jobsheet-01 documentation).
- `Username : [ _____ ]` and `Password : [ _____ ]` → these will become `<label>` + `<input>` pairs, exactly the pattern from the "Add Book" form you already studied in the jobsheet-01 documentation §4.3, except that `Password` will use `type="password"` (a new

input type not used in jobsheet-01 to jobsheet-03, so that the typed characters are hidden as dots).

- [Sign In] → a submit button, like the `<button type="submit">Save</button>` you already know from the Add Book/Member forms.

2.3 A More Complex Wireframe: The Librarian Dashboard

SIMPUS-Mini	Home	Books	Members	Loans	(Name)	Logout
[Total Books]	[Total Members]	[On Loan]				
Quick Actions:						
[+ New Loan]	[+ Return]					
Recent Transactions						
Member	Book	Loan Date	Status			

Look at the navigation row at the very top: `Home | Books | Members | Loans`, this looks very much like the navbar that **already exists** in `index.html` today, merely **with one new menu item added** ("Loans") and a login status indicator on the right (`(Name) Logout`). The row `[Total Books] [Total Members] [On Loan]` likewise describes exactly the statistic cards you **have already built** since jobsheet-02 using CSS Grid (see the jobsheet-02 documentation).

The Dashboard wireframe is deliberately designed to **continue** the elements that already exist (navbar, statistic cards), with new parts added (`Quick Actions`, `Recent Transactions`). This relationship is discussed in more detail in chapter 5.

2.4 Why Are Wireframes Deliberately Plain?

Wireframes are deliberately kept **plain** (no colour, no fonts, no precise measurements) so that the design discussion focuses on **structure and flow** first, which elements need to exist and roughly where they sit, without getting stuck arguing about button colours or typefaces at a very early stage. Visual details like those **already have their answer** when the time comes to implement, because they simply follow the `style.css` you have already built (jobsheet-02 documentation), in line with the note in `wireframe.md` under "Consistency with the Existing Design".

3. User Flow: Borrowing & Returning

Where a wireframe (chapter 2) describes a **single page**, a **user flow** describes the **sequence of steps** a user takes across several pages or actions in order to complete one particular task.

3.1 What Is a User Flow?

A **user flow** is a simple diagram of boxes joined by arrows **->**, showing the **step-by-step order**. Each box represents one screen, one action, or one decision made by the user. Its purpose: to make sure **no step is missing or ambiguous** before the feature is actually coded.

3.2 User Flow: Borrowing a Book

```
[Librarian Login] -> [Dashboard] -> [Select "New Loan" menu]
    -> [Choose Member] -> [Choose Book (stock > 0)]
    -> [Save] -> [Book stock decreases by 1] -> [Back to Dashboard]
```

Let's walk through it box by box:

1. **[Librarian Login]**: the librarian must log in first (the wireframe for this page was covered in chapter 2 §2.1). This box makes it clear that the loan feature **cannot be reached by a Guest**, actors are discussed in more depth in chapter 4.
2. **[Dashboard]**: after a successful login, the librarian lands on the Dashboard page (its wireframe is in chapter 2 §2.3).
3. **[Select "New Loan" menu]**: the librarian clicks one of the "Quick Actions" buttons on the Dashboard.
4. **[Choose Member]** and **[Choose Book (stock > 0)]**: two consecutive form-filling steps. Note the important annotation **(stock > 0)**, this is a **business rule** deliberately written into the user flow: a book with no stock left **must not appear** as an option. Writing this rule down at the design stage makes sure the developer does not forget to apply it later when coding (remember the importance of designing first, from chapter 1 §1.2).
5. **[Save]**: the librarian presses the submit button (much like the `<button type="submit">` you already know from the jobsheet-01 documentation, except that this form will **really be processed** into a database, unlike the Add Book/Member forms in jobsheet-01 to jobsheet-03, which were deliberately left unprocessed).
6. **[Book stock decreases by 1]**: this is an **automatic side effect** behind the scenes: as soon as the loan is saved, the system must reduce that book's stock figure. This box is **not** a page, but a reminder that there is **program logic** to be written behind the Save button (to be implemented with JavaScript/a backend in later jobsheets, see this jobsheet's README.md).
7. **[Back to Dashboard]**: the flow closes by returning to its starting point, showing that the cycle is complete and the librarian can repeat it for the next loan.

3.3 User Flow: Returning a Book

```
[Dashboard] -> ["Return" menu] -> [Find active transaction (member/book)]
```

```
-> [Mark as "Returned"] -> [Book stock increases by 1]
-> [Back to Dashboard]
```

This flow has a similar shape, but note the differences:

- It starts by **finding an active transaction** (a loan that has not yet been returned), rather than filling in a new record from scratch, which makes sense, because returning a book means **matching** it against existing loan data, not creating new data.
- Its side effect is the **opposite** of the borrowing flow: stock **increases** by 1 rather than decreasing.

Comparing two similarly structured user flows like this is a good way to check the **consistency of the design**, if the two flows looked very different with no clear reason, that could be a sign that something needs rethinking.

3.4 Why Write the User Flow Before the Form Exists?

For the same reason given in chapter 1 §1.2: writing down this sequence of steps **before** a single line of the Loan form is coded makes sure the important questions are answered first, for example:

- Can a member borrow **more than one book** in a single transaction? (The Loan form wireframe in [docs/wireframe.md](#) shows one book per transaction, a design decision already taken at this stage.)
- Who is allowed to use this feature?
- What is the most natural order from the point of view of the librarian who actually uses it every day?

4. Actors & Their Link to Authorisation

4.1 What Is an "Actor" in UI/UX Design?

An **actor** is the term for a **type of user** who interacts with the system, distinguished by **what they are allowed to do**. [docs/wireframe.md](#) defines 2 actors for SIMPUS-Mini:

- **Guest:** *can only view the book catalogue (Home, Book List) without logging in.*
- **Librarian:** *logs in to access all CRUD features and loan transactions.*

Defining the actors early matters, because **the very same page** can mean different things depending on who opens it, and that determines which features need to be built for each of them.

4.2 The "Guest" Actor, What You Can Already Try

All the pages that **already exist** from jobsheet-01 to jobsheet-03, Home, Book List and the rest, actually **represent the Guest actor's point of view**: anyone can open them directly without logging in at all. Notice that those pages have no "logout" element or user name anywhere, consistent with the definition of the Guest actor, who "can only view the book catalogue... without logging in."

4.3 The "Librarian" Actor: Newly Designed in This Jobsheet

All the new wireframes in [docs/wireframe.md](#) (Login, Dashboard, the Loan/Return forms, History) take the **Librarian** actor's point of view, and **all of them** require logging in first. This is clear from the user flows discussed in chapter 3: both the Borrowing and the Returning flow **always start** from a box that assumes the librarian is already in the system ([\[Librarian Login\]](#), or straight to [\[Dashboard\]](#), which can only be reached after logging in).

4.4 What Is Authorisation?

Authorisation is the system mechanism that **restricts** who may do or see what, for example, making sure that only a logged-in librarian can open the Loans page, while a Guest who tries to open it is redirected to the Login page.

Note that authorisation needs **real program code** to work (checking login status, storing user sessions and so on), something that static HTML/CSS, as studied in jobsheet-01 to jobsheet-03, cannot yet do. This is exactly why the Login and Loan features are **not implemented** in this jobsheet, as noted in this jobsheet's README.md, these features will only be built in upcoming jobsheets with the help of JavaScript, and later PHP/PostgreSQL to manage real data on the server side.

4.5 Why Define Actors First?

By defining these 2 actors **before** writing any Login code, the developer already has clear answers to the questions that will come up during coding, for example:

- Which pages need to be guarded by a login check, and which may stay freely accessible? (The answer is already implied by the split between actors: catalogue pages for Guests stay open, transaction pages for Librarians need guarding.)

- What happens if a Guest tries to reach the Dashboard URL directly without logging in? (This is an "edge case" that needs thinking about at the design stage, even though its technical implementation comes later.)

5. Connections to the Existing Code

This chapter links the wireframe designs in `docs/wireframe.md` to the HTML/CSS code that has **actually been running** since jobsheet-01 through jobsheet-03, so that you can see this design is not a separate project, but a **direct continuation** of what you have already built.

5.1 The Official Note from `wireframe.md`

The `wireframe.md` document itself closes with a section titled "Consistency with the Existing Design":

- *The accent colours, navbar typography and table/card styles follow the `assets/css/style.css` built up since Jobsheet 2-3.*
- *The navbar will gain a **Loans** menu item and a login status indicator (librarian name / Logout button) from the implementation in Jobsheet 10.*
- *Edge cases to handle during implementation: a book with no stock must not be selectable in the loan form; members with overdue loans are validated in Jobsheet 12 (independent assignment).*

Let's take each of these points in turn.

5.2 Colours & Styles Stay the Same, No Need to Build CSS from Scratch

From jobsheet-02, we have already built:

- The blue theme colour `#1d5b8a` for the header, section headings and submit buttons (see the jobsheet-02 documentation §3.2).
- The white card style with a soft shadow for `<section>` (see the jobsheet-02 documentation §5.3).
- The table style with a coloured header and alternating rows (see the jobsheet-02 documentation §7).
- The form style with bold labels and neat inputs (see the jobsheet-02 documentation §8).

All the new page wireframes (Login, Dashboard, the Loan/Return forms) are **deliberately designed to follow the same patterns**, which means that when these pages really are coded, you **will not need to write the CSS from scratch**. The Login form, for instance, will use a `<label> + <input>` pattern **identical** to the Add Book form you already studied in the jobsheet-01 documentation, so it automatically inherits the tidy styling from `form label` and `form input` in `style.css` without any new CSS at all.

5.3 The Navbar Will Gain a Menu, but the CSS Pattern Stays

Remember from the jobsheet-02 and jobsheet-03 documentation that the navbar is built with Flexbox (`header nav ul { display: flex; ... }`), which lays out **any number of menu items** automatically in a horizontal row (or vertically on mobile through the checkbox hack). This means that adding one new `<li><a>` for the "Loans" menu in Jobsheet 10 **will require no CSS changes whatsoever**, just one extra line of HTML, and Flexbox will rearrange the whole menu automatically. This is a concrete

example of the benefit of designing a generic CSS system from the start (remember the similar note about generic selectors in the jobsheet-02 documentation §6.7).

The same goes for the login status indicator ((`Librarian Name`) `Logout`) visible in the Dashboard wireframe, this will be a new extra element inside `<header>`, most likely laid out alongside `<h1>` and `<nav>` using the same `justify-content: space-between` that keeps `<h1>` and `<nav>` apart today (see the jobsheet-02 documentation §4.4).

5.4 The Same Statistic Cards, a Different Context

The Librarian Dashboard wireframe shows a row of `[Total Books]` `[Total Members]` `[On Loan]`, compare that with the statistic cards that **already exist and work** in `index.html` today (see the full explanation in the jobsheet-01 documentation, and how they are styled with CSS Grid in the jobsheet-02 documentation). This is no coincidence, the Librarian Dashboard design **reuses** exactly the same statistic card component, simply in a different page context (specifically for a logged-in librarian, rather than the public Home page).

5.5 Edge Cases Already Recorded

Two "edge cases" (special or exceptional situations) are already recorded in `wireframe.md`:

1. **A book with no stock must not be selectable** in the Loan form, this also appears in the user flow (chapter 3 §3.2, the `(stock > 0)` annotation). The stock data **already exists** as far back as jobsheet-01: see the "Stock" column in the Book List table, later it will simply need a `stock > 0` check when displaying the book options in the Loan form.
2. **Members with overdue loans**, this case is flagged to be handled in "Jobsheet 12 (independent assignment)", showing that not every detail has to be resolved **all at once**; recording it at the design stage makes sure the case is not forgotten even though its implementation is deferred to another jobsheet.

6. Summary & Exercises

6.1 Summary of Jobsheet 4

Section	Concepts Learned
Core UI/UX Concepts	The difference between UI and UX, why we design before coding
How to Read a Wireframe	The conventions of ASCII wireframe symbols, translating them into HTML elements
User Flow	Step-by-step flow diagrams, recording business rules at the design stage
Actors & Authorisation	The concept of actors (Guest vs Librarian), an introduction to authorisation
Connections to the Code	How the new design reuses the CSS and patterns built since jobsheet-02/03

6.2 Core Concepts to Remember

1. **A wireframe answers "what is on it", a user flow answers "in what order"**, the two complement each other rather than replacing each other (chapters 2 and 3).
2. **Defining the actors up front** helps determine which pages need authorisation guarding, long before the login-checking code itself is written (chapter 4).
3. **A good design reuses** what already exists (CSS, component patterns) rather than building from scratch, visible in the way the Dashboard wireframe adopts the statistic cards and navbar you have been building since jobsheet-02 (chapter 5).
4. **Recording edge cases at the design stage** (a book out of stock, a member with overdue loans) stops those cases from being forgotten during the real coding (chapter 5 §5.5).

6.3 How to Try This Jobsheet

Since there is no new code to run in the browser, the best way to practise this jobsheet is to **reread the wireframes while imagining yourself as the Librarian** carrying out each step:

1. Open `docs/wireframe.md` and read each wireframe while picturing how it would look if it were really drawn following the existing `style.css`.
2. Follow the Borrowing user flow (chapter 3 §3.2) step by step, imagining which page is open at each box.
3. Compare the navbar and statistic cards in the working `index.html` with the Librarian Dashboard wireframe, find for yourself which parts are exactly the same and which are new.

6.4 Exercises

1. **Draw a wireframe for a new page** using the same ASCII conventions (chapter 2 §2.2), for example a "Register New Member" page for the Guest actor who wants to join the library.
2. **Create a new user flow** for a scenario not yet drawn in `wireframe.md`, for example: "The librarian searches for members whose loans are past their due date."

3. **Identify additional edge cases** that may not have been recorded yet, for example: what happens if the librarian tries to lend the same book to the same member twice in a row?
4. **Try implementing the Login wireframe as static HTML** (with no real login logic, much like the unprocessed Add Book form in jobsheet-01) as practice in translating a wireframe into real code, use the `<label> + <input>` pattern you already know from the jobsheet-01 documentation, plus one new `<input type="password">` for the Password field.